Design Rationale

REQ 4: Weapon Capabilities, Coatings, and Unarmed Combat

A. Implementation of Weapon Coating

Design 1: Implement the scanning logic directly inside the Sword class's allowableActions() method. The Sword actively scans the player's inventory for coating items (like Melon and Fire) and the surrounding locations for environmental coatings (like Puddle), then generates the CoatWeaponAction itself.

Pros	Cons
Centralized Logic: All actions related to the sword (attacking and coating) are located within the Sword class, making the execution flow straightforward to read for beginners.	Feature Envy and High Coupling: The Sword class becomes tightly coupled to concrete classes like Melon and Fire. This violates the Open-Closed Principle (OCP); adding any new coating item in the future would require modifying the Sword class
	Overloaded Responsibilities: Weapons should not be responsible for scanning the environment. This design forces the Sword to handle extra responsibilities, violating the Single Responsibility Principle (SRP).

Design 2: Utilize Inversion of Control. The responsibility of generating coating actions is shifted back to the coating sources. Melon and Fire override their allowableActions() to actively scan the actor's inventory for weapons implementing the Coatable interface. For electrified environments, a new abstract class ChargeableGround is created (implementing the Chargeable interface) to handle the inventory scanning and action generation. The concrete Puddle class simply extends this abstract class.

Pros	Cons
Strict Adherence to OCP and DIP: The Sword is completely decoupled from concrete items, depending only on the Coatable interface. New weapons or coating items can be added without modifying existing code (Dependency Inversion Principle).	Segmented Action Flow: The responsibility for generating actions is distributed across multiple classes (items and grounds), which requires a deeper understanding of the engine's menu generation mechanics to trace the code.
Eliminates Code Duplication (DRY): By using the ChargeableGround abstract class, the scanning	

logic is written once and inherited by its subclasses. The Puddle class remains clean and focused only on its specific traits.	

B. Implementation of Unarmed Combat

Design 1: Implement the scanning logic within the Farmer class. When it is the Farmer's turn, the class scans adjacent locations for HostileCreature instances and generates an AttackAction representing a punch.

Pros	Cons
Intuitive Subject-Object Flow: Since the player is the attacker, having the player's class generate the attack action aligns with common logical thinking.	Breaks Double Dispatch Architecture: The engine relies on targets providing allowable actions to the actor. Having the Farmer generate its own menu options bypasses this core engine design.
	God Class Tendency: The Farmer class would need to know the states and types of all potential targets, leading to bloated code that is hard to maintain and extends poorly.

Design 2: Follow the engine's Double Dispatch mechanism. The HostileCreature class overrides allowableActions(Actor otherActor, String direction, GameMap map). When a Farmer is nearby, the monster calls otherActor.getIntrinsicWeapon() to retrieve the farmer's default attack (punch) and returns an AttackAction targeting itself. The Farmer class simply overrides getIntrinsicWeapon() to define the punch's statistics.

Pros	Cons
High Reusability and Consistency: Unarmed combat seamlessly integrates with the engine's existing IntrinsicWeapon (Natural Weapon) system. Both the farmer's punch and a monster's basic attack share the same underlying combat resolution, reducing code duplication (DRY).	Requires Paradigm Shift: Generating an attack action from the perspective of the "target" rather than the "attacker" requires developers to thoroughly understand object-oriented responsibility assignment.
Excellent Encapsulation (SRP): The target (monster) is the information expert on whether it can be attacked. The Farmer only defines its weapon stats (e.g., 5 damage, "punches"), ensuring classes manage only their respective responsibilities (Single Responsibility Principle)	

C. Weapon Class Hierarchy

Design: Instead of extending the base Item class, the Sword class extends a dedicated abstract weapon class (e.g., WeaponItem) or implements the Weapon interface.

Pros	Cons
Promotes Code Reuse: All weapons can share the base attack logic, damage attributes, and hit rate encapsulation provided by WeaponItem, minimizing redundant code across different weapon classes (DRY).	Deeper Inheritance Tree: Introducing an additional layer of abstraction adds slight structural complexity, which might seem redundant for very simple items.
Supports Polymorphism: Other systems can safely use instanceof Weapon to verify an item's identity rather than checking for specific concrete classes. This adheres to the Liskov Substitution Principle.	

Final Design Summary

After evaluating the alternatives, Design 2 was selected across all implementations in REQ 4 to strictly adhere to SOLID principles and the engine's object-oriented architecture.

For the Weapon and Coating module, we removed the hard-coded environmental scanning from the Sword class to eliminate Feature Envy. By introducing the Coatable interface, we inverted the control: Melon and Fire now actively seek out coatable targets. This ensures concrete items depend on abstractions, satisfying the Dependency Inversion Principle. Furthermore, handling electrified terrains through the ChargeableGround abstract class provides a robust Template Method approach. It prevents code duplication in the Puddle class (DRY) and allows future terrains (e.g., metal floors) to inherit coating capabilities effortlessly, strictly following the Open-Closed Principle.

For Unarmed Combat, the architecture embraces the engine's Double Dispatch mechanism. Rather than bloating the Farmer class into a God Class responsible for detecting targets, the HostileCreature acts as the information expert, autonomously offering an AttackAction when approached. This seamlessly ties into the engine's IntrinsicWeapon system. Whether it is a monster's innate strike or the farmer's punch, the combat logic is unified under the Natural Weapon framework. This separation of concerns ensures that the Farmer only dictates what its natural weapon is, while the engine and target handle how it is used, vastly improving the system's cohesion, scalability, and maintainability.